

7.5 배운 내용 정리

정명훈

참고:파이썬 코드 구현 내용은 파이썬 파일을 첨부에 놓았으니 그걸 보시면 됩니다.

오늘은 동적분석과 정적분석을 동시에 이용해서 리버싱의 flag checker문제를 푸는 방식에 대해서 간략히 배웠다

우선 리버싱의 순서는

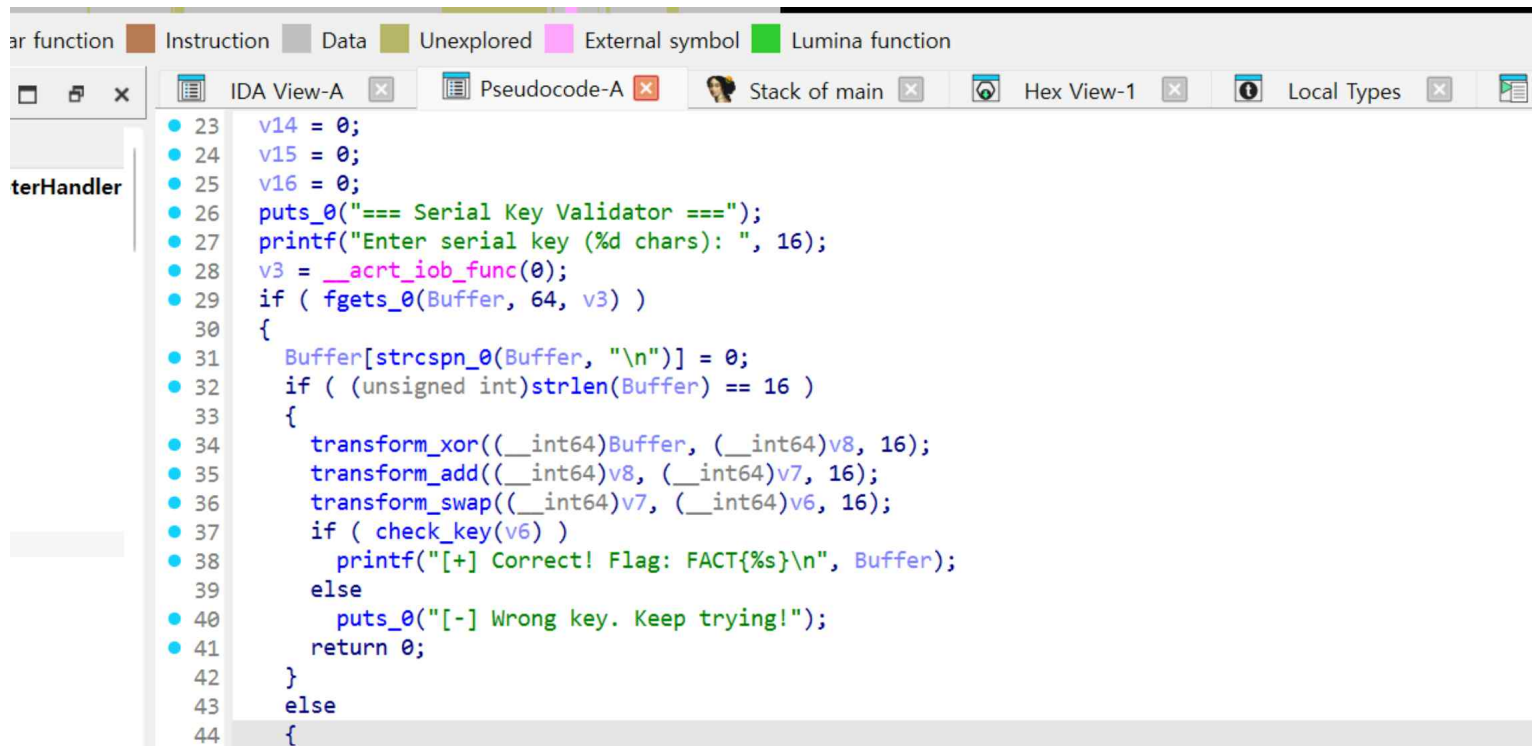
- 1.실행파일을 cmd창에서 한 번 실행시킨다 -> 간단하게 유용한 정보를 획득할 수 있다
- 2.ida에 실행파일을 올린 이후에 정적인 분석을 한다.
3. 정적분석 도중에 내부 함수가 복잡하거나 중간에 멈추는 경우에는 x64dbg를 이용해서 동적인 분석을 한다.
4. 분석을 통해 이 문제가 선형인지 아니면 비선형인지를 알아내야 한다
 - 5.1 선형인 경우에는 역함수(역연산)을 파이썬이나 다른 프로그램을 이용해 구하고 그걸 이용해 정답을 쉽게 도출할 수 있다
 - 5.2 비선형인 경우에는 완전 탐색(브루토포스) 방식을 이용해 전수 조사하는 방식으로 문제를 해결할 수 있다.

여기서 중요한 사실은 1.보통은 main에서 대부분 함수가 실행되는 건 맞지만 아닌 경우도 존재한다는 걸 인지해야한다.

2. main 내 복잡한 함수들 함수 내 함수 등 프로그램이 복잡해질수록 동적분석의 breakpoint를 적절히 활용해 함수의 기능에 대한가설을 세우고 그 가설을 검증하는 과정을 거치는 게 정적 분석하는 것보다 빠르기 때문에 매우 중요해진다는 거다.

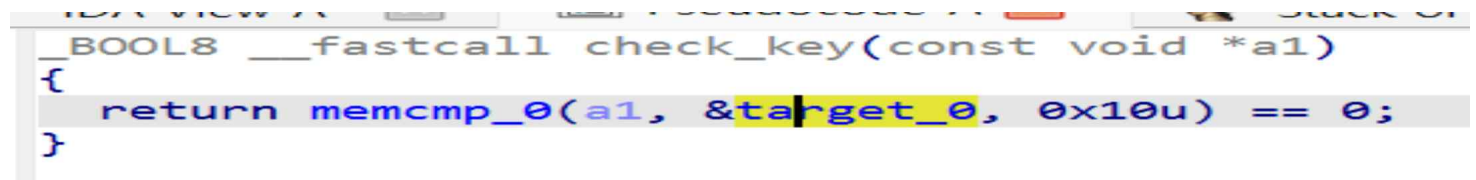
아래에는 실제 내가 배운 걸 이용해 문제를 푸는 과정을 보여주겠다.

1. 우선 challenge.exe파일을 ida에 올려서 정적분석을 먼저한다. main부분을 보면



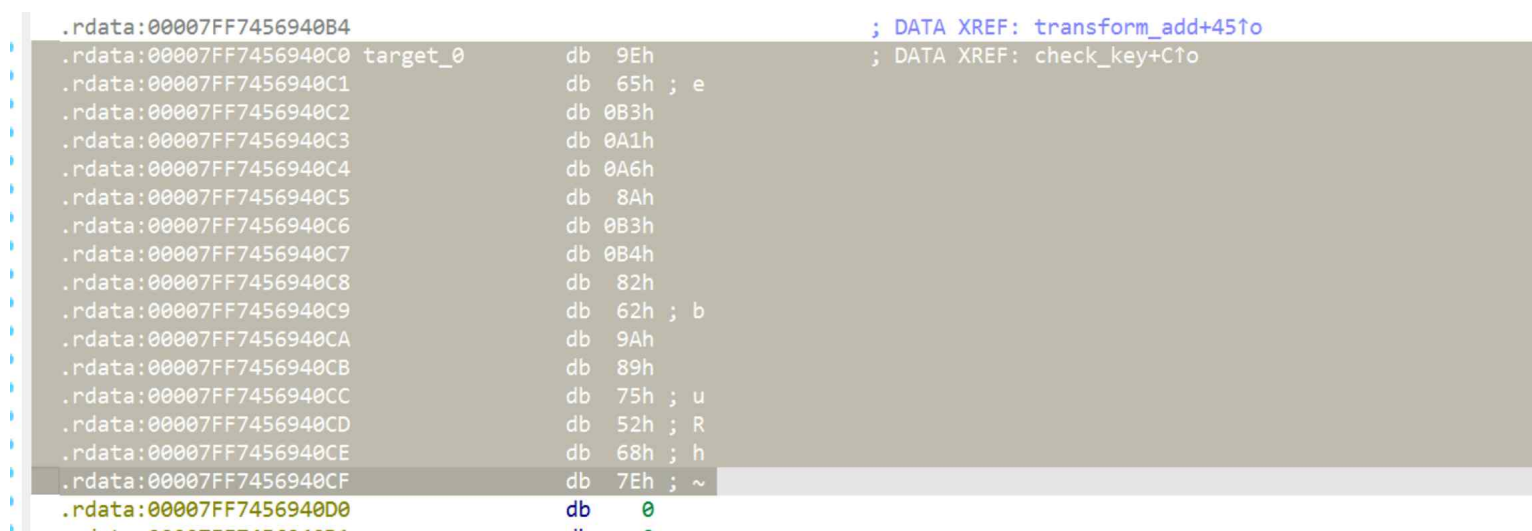
```
ar function Instruction Data Unexplored External symbol Lumina function
IDA View-A Pseudocode-A Stack of main Hex View-1 Local Types
terHandler
23 v14 = 0;
24 v15 = 0;
25 v16 = 0;
26 puts_0("=== Serial Key Validator ===");
27 printf("Enter serial key (%d chars): ", 16);
28 v3 = __acrt_iob_func(0);
29 if ( fgets_0(Buffer, 64, v3) )
30 {
31     Buffer[strcspn_0(Buffer, "\n")] = 0;
32     if ( (unsigned int)strlen(Buffer) == 16 )
33     {
34         transform_xor((__int64)Buffer, (__int64)v8, 16);
35         transform_add((__int64)v8, (__int64)v7, 16);
36         transform_swap((__int64)v7, (__int64)v6, 16);
37         if ( check_key(v6) )
38             printf("[+] Correct! Flag: FACT{%s}\n", Buffer);
39         else
40             puts_0("[-] Wrong key. Keep trying!");
41         return 0;
42     }
43     else
44     {
```

이 그림에서 결국 원하는 flag를 얻기 위해서는 check_key함수가 가장 중요하다는 걸 알 수 있다. 따라서 check_key함수 안으로 들어가 보면



```
__BOOL8 __fastcall check_key(const void *a1)
{
    return memcmp_0(a1, &target_0, 0x10u) == 0;
}
```

내가 입력한 16자리 값이 target0과 같은지 비교해서 맞는 경우에는 correct가 뜬다는 거다 그러면 결국 나는 target0에 들어가서 목표값을 일단 한 번 확인해야 한다.



```
.rdata:00007FF7456940B4 ; DATA XREF: transform_add+45f0
.rdata:00007FF7456940C0 target_0 db 9Eh ; DATA XREF: check_key+Cf0
.rdata:00007FF7456940C1 db 65h ; e
.rdata:00007FF7456940C2 db 0B3h
.rdata:00007FF7456940C3 db 0A1h
.rdata:00007FF7456940C4 db 0A6h
.rdata:00007FF7456940C5 db 8Ah
.rdata:00007FF7456940C6 db 0B3h
.rdata:00007FF7456940C7 db 0B4h
.rdata:00007FF7456940C8 db 82h
.rdata:00007FF7456940C9 db 62h ; b
.rdata:00007FF7456940CA db 9Ah
.rdata:00007FF7456940CB db 89h
.rdata:00007FF7456940CC db 75h ; u
.rdata:00007FF7456940CD db 52h ; R
.rdata:00007FF7456940CE db 68h ; h
.rdata:00007FF7456940CF db 7Eh ; ~
.rdata:00007FF7456940D0 db 0
```

확인하면 이렇게 나오고 이걸 나중에 역연산 혹은 브루토포스 방식 이용시에 활용할 아주 귀중한 자료이다.

그 다음에는 transform_xor,add,swap함수를 분석해서 우선 이것과 똑같은 기능을 하는 함수를 파이썬이나 다른 프로그램을 이용해서 구현해야 한다. 우선 3개다 확인한 결과

3개다 함수 내 함수(서브루틴)은 없는 비교적 간단한 함수였고 사실 이정도면 그냥 셋다 정적 분석으로 정연산을 구현해도 되지만 동적 분석 연습을 위해 swap함수만 동적으로 기능을 파악해보기로 결정했다.

우선xor함수를 보면 말 그대로 __int64 __fastcall transform_xor(__int64 a1, __int64 a2, int a3)

```
{  
  
    __int64 result; // rax  
  
    int i; // [rsp+Ch] [rbp-4h]  
  
    for ( i = 0; ; ++i )  
    {  
  
        result = (unsigned int)i;  
  
        if ( i >= a3 )  
  
            break;  
  
        *(_BYTE *)(i + a2) = (i + 1) ^ *(_BYTE *)(i + a1);  
  
    }  
  
    return result;  
}
```

} 입력한 값에 대해서 $a2[i] = (i+1) \wedge a1[i]$ 와 같은 변형을 가해주는 함수로 이를 파이썬 함수를 이용해 구현합니다.(구현은 따로 파이썬 파일에 저장해 놔습니다)

그 다음에는 transform_add를 보면__int64 __fastcall transform_add(__int64 a1, __int64 a2, signed int a3)

```
{  
  
    __int64 result; // rax  
  
    signed int i; // [rsp+Ch] [rbp-4h]  
  
    for ( i = 0; ; ++i )
```

```
{
    result = (unsigned int)i;

    if ( i >= a3 )

        break;

    *(_BYTE *)(i + a2) = magic_1[i % 4] + *(_BYTE *)(i + a1);

}

return result;
```

} magic_1배열의 값을 더해주고 있음을 알 수 있습니다. 이 값의 위치를 찾기 위해서는 더블클릭하고

```
.rdata:00007FF7456940B4 ; _BYTE magic_1[12]
.rdata:00007FF7456940B4 magic_1 db 13h, 37h, 41h, 42h, 8 dup(0)
```

이 값을 선택하고 그 다음에 edit -> export_data로 13374142를 얻어서 그걸 이용해서 파이썬에서 table = bytes.fromhex('13374142')와 같이 가져와서 연산 과정을 구현해 주면 된다 그 다음에 swap함수는 동적 분석을 통해서 알아내려고 하는데 우선 그러기 위해서는 ida의 주소와 xdbg의 주소를 일치시키고 그 다음에 ida의 swap함수 주소를 이용해 xdbg에서 swap함수를 찾아야 한다 우선 base주소를 일치시키기 위해서는 먼저 f9를 눌러 실행시킨뒤 메모리 앱에 가서

주소	크기	파티	정보	내용	유형	보호	초기
000000007FFE0000	0000000000001000	사용자	KUSER_SHARED_DATA		PRV	-R---	-R---
000000007FFE2000	0000000000001000	사용자	예약됨		PRV	-R---	-R---
0000000048740000	0000000000015C00	사용자	예약됨		PRV	-R---	-R---
0000000048752C00	0000000000009000	사용자	PEB, TEB (11204), TEB (17936)		PRV	-RW--	-RW--
0000000048753500	000000000000B000	사용자	예약됨 (0000006487400000)		PRV	-RW--	-RW--
0000000048760000	000000000001F800	사용자	예약됨		PRV	-RW--	-RW--
000000004877F800	0000000000008000	사용자	스택 (11204)		PRV	-RW-G	-RW--
0000000048780000	000000000001FC00	사용자	예약됨		PRV	-RW--	-RW--
000000004879FC00	0000000000004000	사용자	스택 (17936)		PRV	-RW-G	-RW--
00000000487A0000	000000000001FC00	사용자	예약됨		PRV	-RW-G	-RW--
00000000487BFC00	0000000000004000	사용자	예약됨		PRV	-RW-G	-RW--
00000000487C0000	000000000001FC00	사용자	예약됨		PRV	-RW-G	-RW--
00000000487DFC00	0000000000004000	사용자	예약됨		PRV	-RW-G	-RW--
000001EA8DE70000	0000000000001000	사용자	예약됨 (000001EA8DE70000)		MAP	-RW--	-RW--
000001EA8DE71000	0000000000001000	사용자	예약됨		MAP	-R---	-R---
000001EA8DE90000	0000000000002A00	사용자	예약됨		MAP	-R---	-R---
000001EA8DE90100	000000000000F000	사용자	예약됨		MAP	-R---	-R---
000001EA8DE90000	0000000000001000	사용자	예약됨		MAP	-R---	-R---
000001EA8DE90000	0000000000002000	사용자	예약됨		PRV	-RW--	-RW--
000001EA8DE90000	0000000000003000	사용자	예약됨		PRV	-RW--	-RW--
000001EA8DF40000	000000000000A000	사용자	예약됨		MAP	-R---	-R---
000001EA8DF50000	000000000000A000	사용자	예약됨		MAP	-R---	-R---
000001EA8DF60000	0000000000007000	사용자	예약됨		MAP	-R---	-R---
000001EA8DF70000	0000000000009000	사용자	예약됨		MAP	-R---	-R---
000001EA8DF80000	0000000000003000	사용자	예약됨		MAP	-R---	-R---
000001EA8DF90000	0000000000003100	사용자	예약됨		MAP	-R---	-R---
000001EA8DF90000	0000000000001000	사용자	예약됨		MAP	-R---	-R---
000001EA8DF90000	0000000000001000	사용자	예약됨		PRV	-RW--	-RW--
000001EA8E010000	000000000000F000	사용자	예약됨		PRV	-RW--	-RW--
000001EA8E0F0000	000000000000B300	사용자	예약됨		MAP	-R---	-R---
00007FF46E780000	0000000000005000	사용자	예약됨		MAP	-R---	-R---
00007FF46E785000	000000000000F800	사용자	예약됨 (00007FF46E780000)		MAP	-R---	-R---
00007FF46E880000	0000000100020000	사용자	예약됨		PRV	-RW--	-RW--
00007FF56E8A0000	0000000000000000	사용자	예약됨		PRV	-RW--	-RW--
00007FF5708A0000	0000000000000100	사용자	예약됨		PRV	-RW--	-RW--
00007FF570880000	0000000000000100	사용자	예약됨		MAP	-R---	-R---
00007FF745690000	0000000000000100	사용자	challenge.exe		IMG	-R---	ERWC-
00007FF745691000	0000000000002000	사용자	challenge.exe		IMG	-R---	ERWC-
00007FF745693000	0000000000001000	사용자	challenge.exe		IMG	-RWC-	ERWC-

challenge.exe파일의 base주소를 찾아 복사한 다음에 ida에 가서 edit -> segments -> rebase_program에 들어가 복사한 주소 앞에 0x를 붙여주면 된다 그 다음에는 ida에서 swap함수의 주소를 복사하고 xdbg로 와서 ctrl + g를 입력후 거기에 붙여넣기 한 다음에 이동을 누르면 swap함수를 찾을 수 있다. 그 다음에는 swap함수 실행 전후 차이를 알기 위해서 breakpoint를 swap함수에 걸고 rip을 그 위에 설정한 다음에 swap된 값이 저장되는 ss:[rbp-70]의 swap함수 전 후 실행값을 좌측 아래 dump값에 추적

을 통해서 확인하면

 덤프 1	 덤프 2	 덤프 3	 덤프 4	 덤프 5	 주시 1	 로컬
주소	Hex				ASCII	
00000064877FF6F0	9A 73 A7 A3	9B 7A AC A6	A0 7D B1 A9	A1 7C B6 AC	.s\$F.z~ }±0; ~	
00000064877FF700	73 9A A3 A7	7A 9B A6 AC	7D A0 A9 B1	7C A1 AC B6	s.f\$z.~} 0± ~	

이렇게 9a 73 이 73 9a로 바뀌듯 양 옆 바이트끼리 서로 바뀌는 함수라는 걸 알 수 있다. 그 점을 이용해서 파이썬으로 정연산을 구현하고 그 다음에는 역연산과 브루토포스 방식으로 정답을 구하면 된다 그 과정은 파이썬 파일에 방법이 적혀있으니 보면 된다.